UT1 · Desarrollo Web en Entorno Servidor

Cloud 9 Systems

Entorno de desarrollo y organización del proyecto

Engineering your cloud.



Yoel Bautista · Ángel León · Juan Millet · Damián Pérez
CIFP Villa de Agüimes · 2º DAW · Curso 2026/27

PARTE 1

Instalación del entorno

Ocho fases, del 25/09 al 04/10/2026, hasta ver Laravel funcionando

1 **Ubuntu**

Operativo el 25/09

2 **VS Code**

Operativo el 26/09

3 **PHP**

Operativo el 28/09

4 **Composer**

Operativo el 29/09

5 **Git y GitHub**

Operativo el 01/10

6 **Node y npm**

Operativo el 02/10

7 **MariaDB**

Operativo el 03/10

8 **Laravel**

Operativo el 04/10

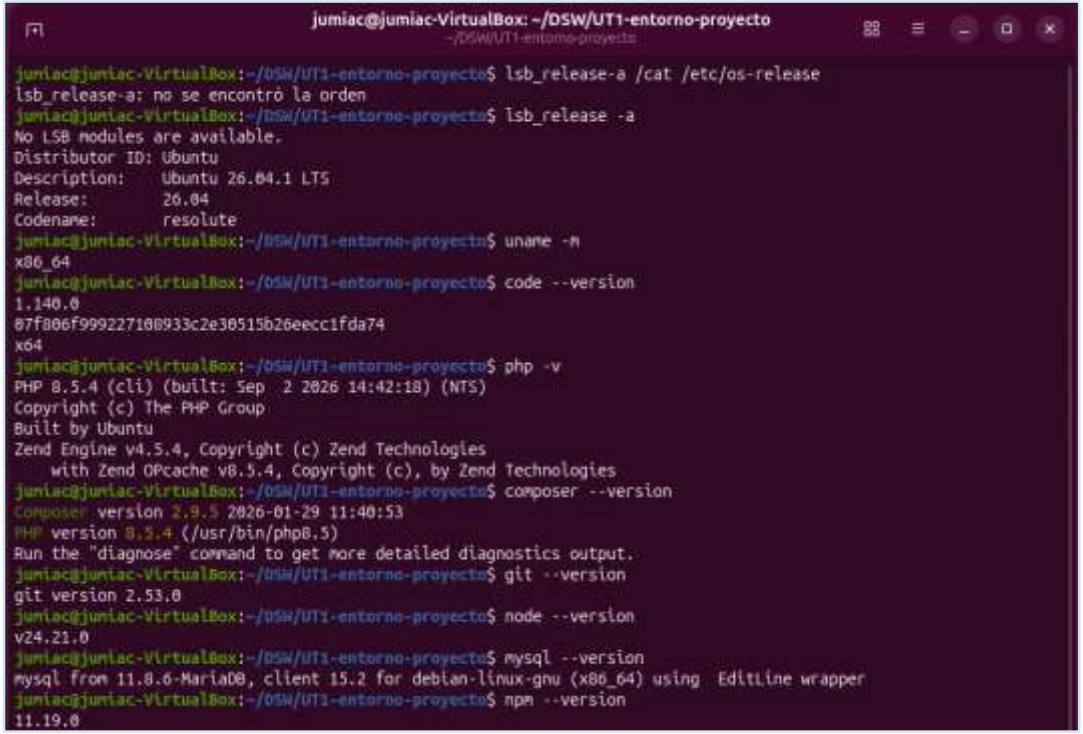
Qué hemos instalado: versiones verificadas

Componente	Versión	Verificación
Ubuntu	**26.04.1 LTS**	`lsb_release -a`
Visual Studio Code	**1.140.0**	`code --version`
PHP	**8.5.4**	`php -v`
Composer	**2.9.5**	`composer --version`
Git	**2.53.0**	`git --version`
Node.js / npm	**v24 / 11.19.0**	`node -v · npm -v`
MariaDB	**11.8.6**	`mysql --version`
Laravel	**13.33.0**	`php artisan --version`

Máquina virtual

- VirtualBox sobre Ubuntu
- 2 CPU · 4 GB de RAM
- Red NAT: 10.0.2.15
- Equipo: jumiac-VirtualBox
- Instalación limpia desde ISO oficial

Verificación desde la terminal



Primer paso: Ubuntu al día

```
sudo apt update
sudo apt upgrade -y
```

Repositorios sincronizados y paquetes actualizados antes de instalar nada más.

Visual Studio Code: el IDE del proyecto

Extensión	Para qué sirve	¿Obligatoria?
PHP IntelliSense	Autocompletado y navegación de código PHP	**Obligatoria**
Laravel Artisan	Ejecuta comandos Artisan desde VS Code	Opcional (recomendada)
Laravel Blade Snippets / Formatter	Resaltado y formateo de vistas .blade.php	**Obligatoria**

Terminal integrada

Lanza php artisan serve, migraciones y genera modelos, servicios y controladores sin salir del editor.

Git integrado

Comandos rápidos en terminal (pull, add, commit) y vista diff para revisar y resolver conflictos antes de subir.

PHP 8.5.4 y Composer 2.9.5

php-mbstring

Texto multibyte: tildes e idiomas

php-xml

Documentos XML para Laravel y Composer

PHP

Lenguaje que se ejecuta en el servidor y entrega HTML al navegador. Laravel es el framework construido encima.

php-mysql

Driver PDO para MariaDB/MySQL

php-curl

Peticiones HTTP externas, p. ej. API de IA

Composer

Gestor de dependencias de PHP: descarga paquetes y los carga con su autoload.

```
Comprobación: php -m | grep mbstring
```

Cadena de dependencia

PHP → Composer → Laravel

Git y GitHub: repositorio compartido

github.com/JuanMiAc/DSW_CLOUD_AJDJ

Rama	Finalidad	Estado
`main`	Código estable de producción	Activa
`rama_develop`	Integración de funcionalidades	En desarrollo
`rama_test`	Pruebas antes de pasar a main	En pruebas
`rama_juan_bd`	Modelado y bases de datos (Juan)	Activa
`feature/entorno`	Configuración inicial de la VM (Damián)	Fusionada y cerrada



Ramas activas en GitHub

Acceso: público · profesor con acceso · 4 miembros

Node.js, npm y Vite: el frontend

Node.js

Laravel corre en PHP, pero el entorno necesita Node para compilar los recursos de la interfaz.

npm

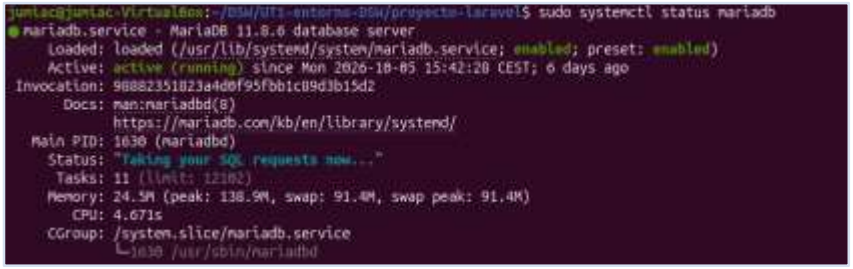
Gestor de paquetes de Node. Instala librerías JavaScript y CSS (Tailwind, Bootstrap...).

Vite

Empaqueta el frontend y recarga el navegador al guardar cambios.

`package.json` lista las dependencias y versiones exactas del frontend. **npm 11.19.0** · Node.js **v24**

MariaDB 11.8.6: base de datos operativa



sudo systemctl status mariadb → active (running)



Puerto 3306 · usuario root@localhost · bases del sistema

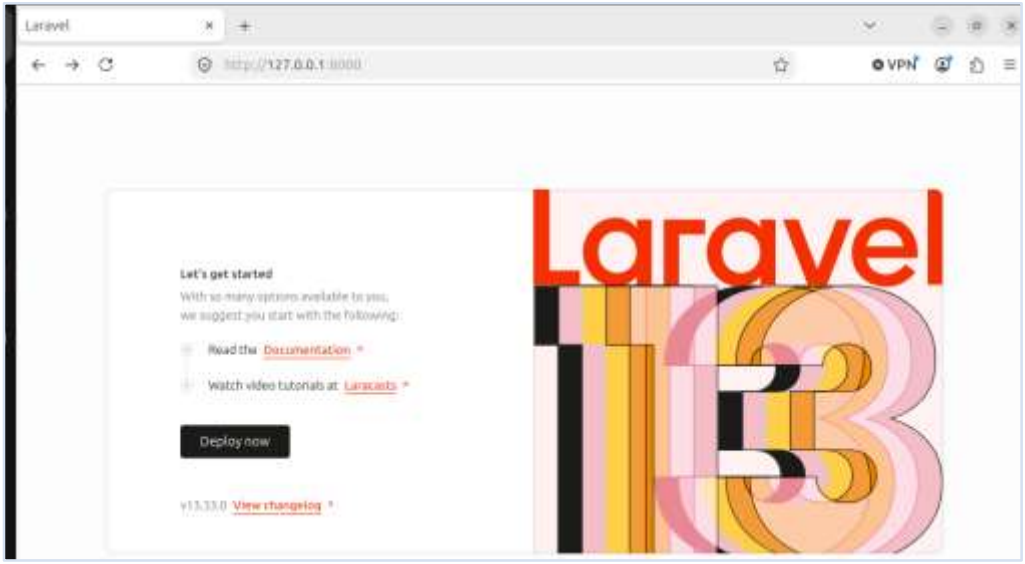
Conexión desde Laravel

Se configura en el fichero .env (driver mysql, puerto 3306). Las tablas se crearán con migraciones.

El fichero .env

Guarda credenciales y claves del entorno local. Cambia entre desarrollo y producción y no se sube a Git.

Laravel 13.33.0 funcionando



Pantalla de bienvenida en http://127.0.0.1:8000



Creación del proyecto

```
composer create-project
laravel/laravel proyecto-laravel
```

Ruta de comprobación /dsw

Ruta → controlador → vista Blade → HTML. Muestra el mensaje «Entorno DSW funcionando correctamente.» con las versiones de PHP y Laravel.

Seguridad inicial del repositorio

.env nunca se sube

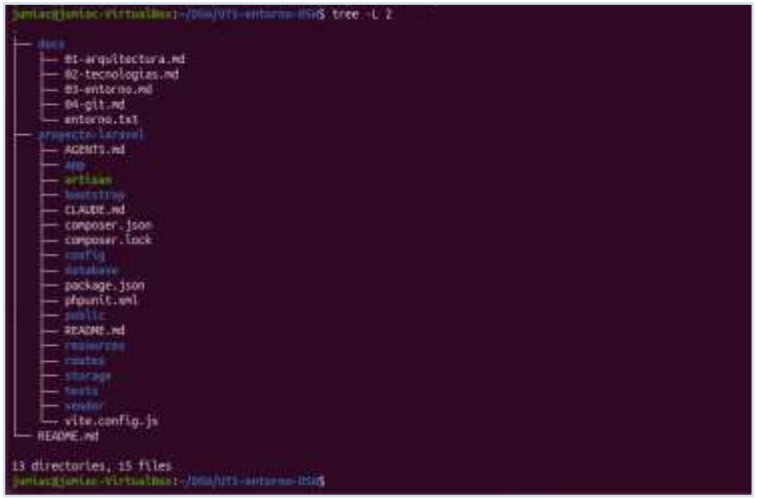
Contiene contraseñas de BD, API keys de la IA y la APP_KEY que cifra sesiones y cookies. Si se filtra, hay accesos falsos y costes por uso fraudulento.

.env.example como plantilla

Solo nombres de variables, sin secretos. Un compañero lo copia a .env y pone sus propios datos.

Comprobación	Resultado
.env está en .gitignore	**Sí**
Sin contraseñas, tokens ni claves privadas en el repo	**Comprobado**
.env.example sin secretos	**Comprobado**

Procedimiento de reproducción



1. sudo apt update && sudo apt upgrade -y
2. sudo snap install --classic code
3. sudo apt install -y php-cli php-mbstring php-xml php-curl php-mysql
4. Instalar Composer en /usr/local/bin/composer
5. sudo apt install git -y
6. sudo apt install nodejs npm -y
7. sudo apt install mariadb-server -y
8. Crear la base de datos proyecto_laravel
9. composer create-project laravel/laravel proyecto-laravel

Incidencias y conclusiones

Incidencia	Causa	Solución
php artisan: «Could not open input file»	Comando lanzado fuera de la carpeta del proyecto	cd proyecto-laravel y repetir
free - h muestra la ayuda	Espacio de más entre guion y parámetro	Escribir free -h
apt bloqueado al actualizar	Actualizaciones automáticas en segundo plano	Esperar unos minutos

El entorno está listo para la UT2

- PHP, Composer, Node, MariaDB y Git verificados por terminal
- Repositorio compartido operativo y ruta /dsw respondiendo en local
- Aún sin funcionalidades: la codificación empieza en la UT2

PARTE 2

Organización del proyecto

Plataforma de simulación de infraestructura Cloud con asistente de IA

El problema

Diseñar y presupuestar una infraestructura Cloud es complejo para usuarios no expertos y PYMEs: recursos, costes y compatibilidad.

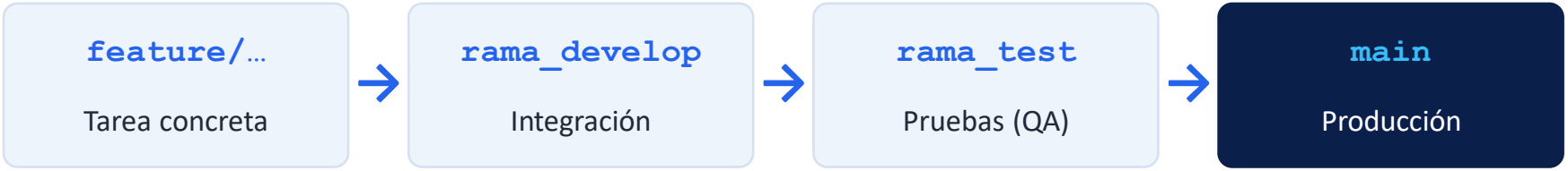
La solución

Una web Laravel que traduce requisitos en una propuesta de servidores, BD, redes y backups, y genera un presupuesto simulado.

La clave

La IA solo recomienda. Laravel valida compatibilidades, aplica tarifas y controla el presupuesto de 50 €/mes.

Cómo trabajamos: ramas, commits y Pull Requests



Tipo	Uso	Ejemplo
`feat`	Nueva funcionalidad	feat(auth): login con dos factores
`fix`	Corrección de un error	fix(user): validar el correo
`docs`	Solo documentación	docs(readme): instrucciones
`refactor`	Mejora sin cambiar el comportamiento	optimizar consulta Eloquent

Reglas de Pull Request

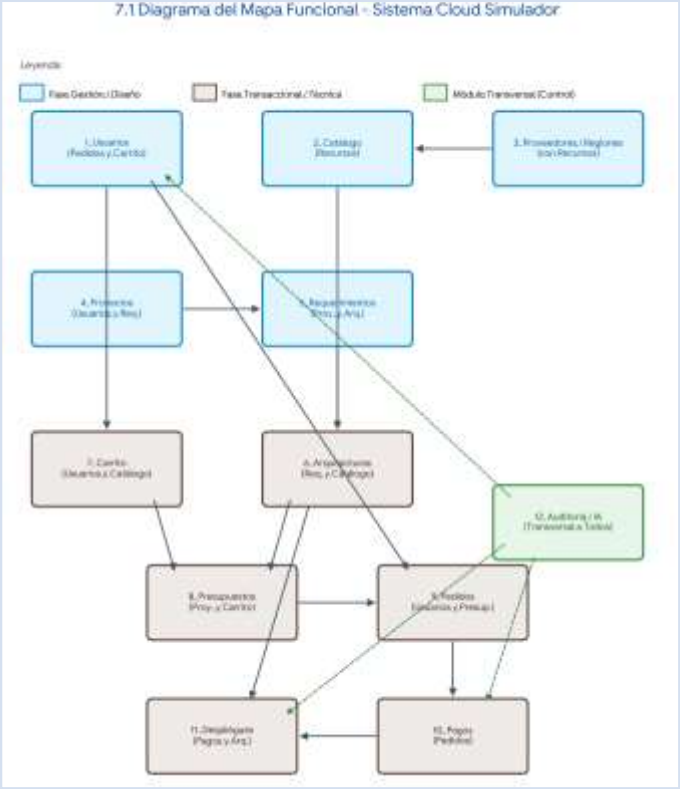
- main y develop protegidas: sin push directo
- Nadie fusiona su propio PR
- Aprobación de otro integrante
- composer audit y npm audit antes de integrar

Los cuatro actores del sistema

Actor	Objetivo	Privilegio exclusivo
Cliente	Pedir presupuestos, seguir sus proyectos y pagar	Único que inicia pagos y valora el servicio
Arquitecto	Diseñar y planificar el desarrollo técnico de los proyectos asignados	Sube planos y diagramas de arquitectura
Auditor	Supervisar procesos, seguridad y cumplimiento	Acceso de solo lectura a logs e informes de cumplimiento
Administrador	Gestionar usuarios, roles y la seguridad de la plataforma	Único que da de alta/baja usuarios y cambia la configuración global

Un usuario solo tiene un rol activo a la vez (control de acceso por roles, RBAC).

Mapa funcional: 12 módulos



Gestión y diseño

1 Usuarios · 2 Catálogo · 3 Proveedores/regiones · 4 Proyectos · 5 Requerimientos

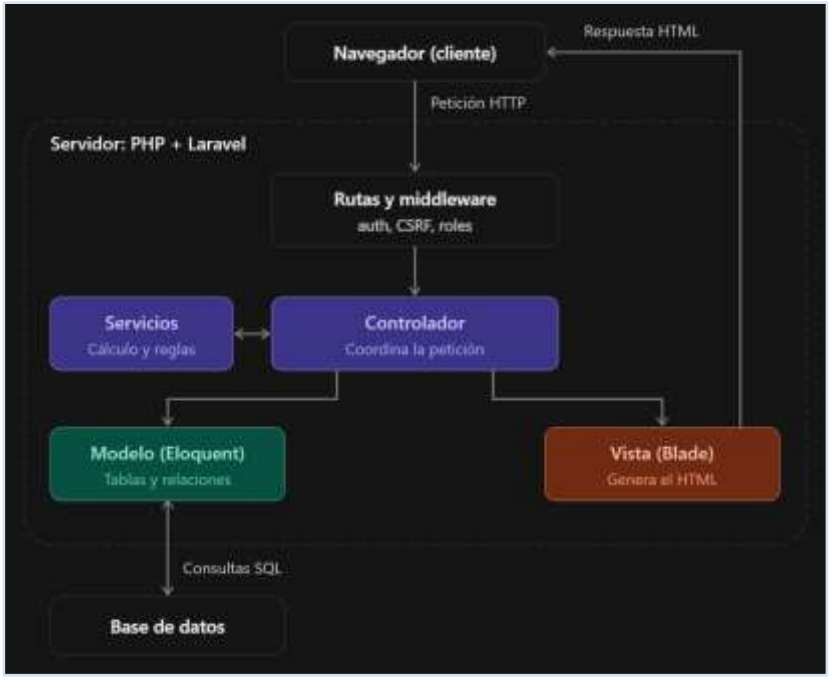
Transaccional

6 Arquitecturas · 7 Carrito · 8 Presupuestos · 9 Pedidos · 10 Pagos · 11 Despliegues

Transversal

12 Auditoría / IA
(registra todo el sistema)

Arquitectura MVC y ciclo de una petición

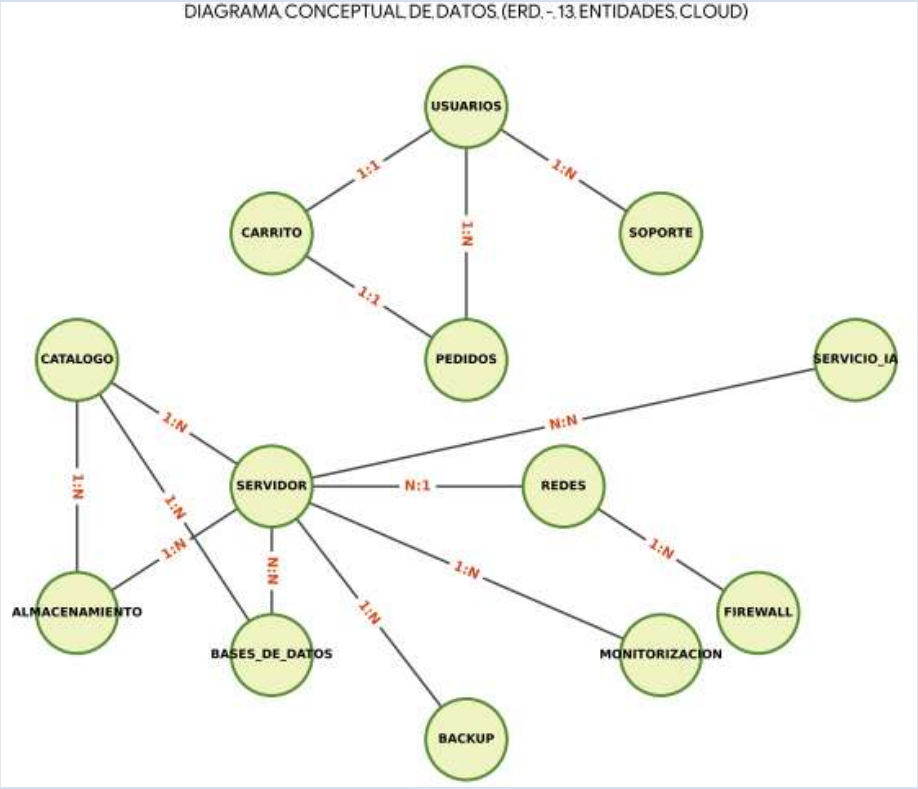


Capas: ruta y middleware → controlador → modelo / servicio / vista



GET / → index.php → autoload → bootstrap → rutas → vista Blade → 200 OK

Modelo conceptual de datos: 13 entidades



Núcleo comercial

Usuarios, Carrito y Pedidos: cuentas, selección temporal de recursos y orden de contratación.

Núcleo técnico

Servidor es el centro: se relaciona con Catálogo, Redes, Almacenamiento, Bases de datos, Backup, Monitorización y Servicio IA.

Riesgos principales y medidas

ID	Riesgo	Prob.	Impacto	Medida
R01	Credenciales o claves en el repositorio	Media	**Alto**	.gitignore, .env.example y revisión de PR
R04	Accesos no autorizados según el rol	Media	**Alto**	RBAC con middleware, Gates y Policies
R06	Vulnerabilidades en dependencias	Media	**Alto**	composer audit y npm audit
R07	Datos de usuario mal tratados	Media	**Alto**	Form Requests, Eloquent, CSRF y XSS
R08	Recomendaciones erróneas de la IA	Media	Medio	Validar la respuesta JSON contra el catálogo real
R10	Pérdida de trazabilidad de cambios	Media	Medio	Audit Log: usuario, acción, IP y valores

Selección de 6 de los 10 riesgos identificados (R01–R10).

Alcance y próximos pasos

Incluido

- Registro de proyectos y catálogo simulado
- Recomendación de IA validada por Laravel
- Presupuestos inmutables con IVA y desglose
- Pago simulado, pedido y panel de despliegue
- Auditoría y control de acceso por roles

Fuera de alcance

- Aprovisionamiento real en AWS, Azure o GCP
- Cobros reales con pasarelas bancarias
- Scripts Terraform o Ansible
- Monitorización en tiempo real
- Migración de datos entre proveedores

UT1 cierra con el entorno validado y la documentación lista. La implementación empieza en la UT2.